

Audit 4

Ben Pritzkau & Robin Schlegel

Demonstration

- Videolink auf Github im Audit4 README

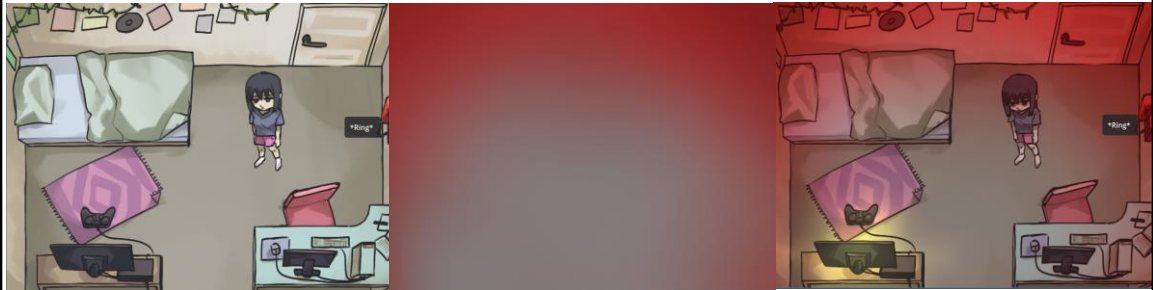
Code Inspektion

- Shader
- Textur wird im shader code mithilfe der `hard_light` funktion auf die Szene angewendet
- Parameter "intensity" steuert wie stark der effekt ist

```
1  shader_type canvas_item;  
2  uniform sampler2D SCREEN_TEXTURE:hint_screen_texture;  
3  uniform float intensity : hint_range(0,1) = 0.0;  
4  
5  vec4 hard_light(vec4 base, vec4 blend){  
6      vec4 limit = step(0.5, blend);  
7      return mix(2.0 * base * blend, 1.0 - 2.0 * (1.0 - base) * (1.0 - blend), limit);  
8  }  
9  
10 void fragment()  
11 {  
12     vec4 bg_color = texture( SCREEN_TEXTURE, SCREEN_UV );  
13     COLOR.rgb = mix(bg_color, hard_light( bg_color, COLOR ), intensity).rgb;  
14 }
```

```
1  extends Sprite2D  
2  @onready var vMan := %"Variablen-Manager"  
3  
4  # Called every frame. 'delta' is the elapsed time since the previous frame.  
5  func _process(delta: float) -> void:  
6      $".".material.set_shader_parameter("intensity", clamp((-vMan.gesundheit + 50.0) * 0.02, 0.0, 1.0))
```

Schauen wir uns zum Beispiel einen der Shader an. Hier zu sehen ist der shader welcher angewandt wird wenn die Gesundheit des Spielers zu gering ist. Dafür haben wir eine Textur erstellt (Hier als die Variable `SCREEN_TEXTURE`), welche auf das Hintergrundbild angewandt werden soll. Das geschieht mittels der `hard_light` funktion, welche Mathematisch identische zu dem hard light Ebenenmodus in Bildbearbeitungs Programmen wie zB. Photoshop aufgebaut ist. "bg_color" ist in diesem fall der Hintergrund auf welcher die Textur dann in zeile 13 angewendet wird zusammen mit der "intensity" variable. Im normalen Code wird dann auf die "intensity" Variable im shader zugegriffen und mittels dem Gesundheits parameter einen wert von 0 bis 1 zugewiesen.

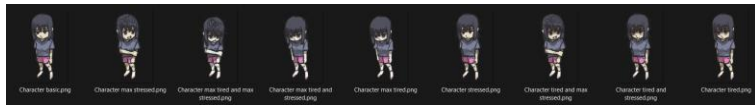


Hier können sie die Overlaytextur sehen und wie sie sich auf das Bild auswirkt

Code Inspektion

- Character Sprites für die verschiedenen Gefühlszuständen.
- Je nach Stress Level werden die gewünschten Sprites angezeigt.

```
9 func _process(delta: float) -> void:
10     if((vMan.stress > 40) && (vMan.gesundheit < 60)):
11         anim.play("tired and stressed")
12     if((vMan.stress > 80) && (vMan.gesundheit < 20)):
13         anim.play("max tired and max stressed")
14     elif ((vMan.stress > 80) && (vMan.gesundheit < 60)):
15         anim.play("tired and max stressed")
16     elif ((vMan.stress > 40) && (vMan.gesundheit < 20)):
17         anim.play("max tired and stressed")
18     elif(vMan.stress > 40):
19         anim.play("stressed")
20     if(vMan.stress > 80):
21         anim.play("max stressed")
22     elif(vMan.gesundheit < 60):
23         anim.play("tired")
24     if(vMan.gesundheit < 20):
25         anim.play("max tired")
26     else:
27         anim.play("default")
```

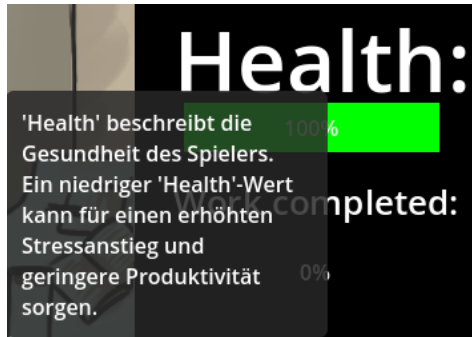


Ein weiteres Beispiel dafür wie die Zustandsparameter mit dem Spiel interagieren sind die Character Sprites. Die Sprites haben wir für unseren spezifischen Kontext selber erstellt. Dabei gibt es einen Sprite für alle verschiedenen Kombinationen von normal, zwei gestresst Zuständen und zwei Gesundheits Zuständen. In der Player Klasse wird nun mittels If-Verzweigungen auf die Zustandsparameter zugegriffen und für die möglichen Zustandskombinationen jeweils der richtige Sprite ausgegeben und angezeigt.

Handlungsstrang

- Kurzer Handlungsstrang
- 2 Studenten im Entwicklungsprojekt
- Partner ist auf einem Ausflug und arbeitet nicht mehr mit
- Erfolg des Projekts beruht auf dem Erfolg des Spielers
- Je nach Arbeitsfortschritt, alternative Dialoge

Tooltips



Fazit

Projektübersicht

- Im Exposé:

„In Folge dieses Projekts, soll ein Spiel-System entwickelt werden, welches den Prozess der Prokrastination, die daraus entstehende Problemlage und, wenn möglich, potenzielle Lösungsansätze illustriert. Dabei soll als Grundlage der Alltagsprozess eines Studenten verwendet werden, in welchem Pflicht und Ablenkungen vorhanden sind.“

In der Projektübersicht definierten wir das Projekt wie folgt:

„In Folge dieses Projekts, soll ein Spiel-System entwickelt werden, welches den Prozess der Prokrastination, die daraus entstehende Problemlage und, wenn möglich, potenzielle Lösungsansätze illustriert. Dabei soll als Grundlage der Alltagsprozess eines Studenten verwendet werden, in welchem Pflicht und Ablenkungen vorhanden sind.“

Projektübersicht – Umsetzung

- Ein Spiel-System wird verwendet
- Durch Implikation:
 - Aufzeigen des Prozesses von Prokrastination
 - Aufzeigen der Problemlage der Prokrastination
 - Aufzeigen potenzieller Lösungsansätze
- Setting -> Alltag eines Studenten, in welche Pflicht und Ablenkungen existieren

Diese Punkte fanden unserer Ansicht nach ihre Umsetzung. Es wird ein Spiel-System verwendet und die Systeme im Spiel verweisen auf implikative Weise auf den Prozess, die Problematik und potenzielle Lösungsansätze für Prokrastination, im Projekt der Aspekt des Zeitmanagements.

Auch der Aspekt des Settings, der Alltag eines Studenten, bestehend aus Pflicht und Ablenkung wurde umgesetzt.

Besser sichtbar wird der Grad der Umsetzung aber durch einen Blick auf die im Exposé geschilderten Mechaniken.

Mechanik Ziele

Ziele im Exposé

- Zeitmanagement
- Stress-/Gesundheitszustand
- Aufgaben(Arbeiten)
- Aktivitäten(Spielen + Schlafen)
- 2 Spielenden
- Visuelle Repräsentation der Parameter
 - „visuelle Effekte wie Licht und Schatte, Inszenierung der Kamera ...“

Umgesetzt

- Zeitmanagement
- Stress-/Gesundheitszustand
- Aufgaben(Arbeiten)
- Aktivitäten(Spielen + Schlafen)
- 2 Spielenden
- Visuelle Repräsentation der Parameter
 - Shader
 - Dynamische Character Sprites
 - Texttransformation
- Story-Line
- Tooltips

Im Exposé setzten wir uns die Ziele, dass das Spiel die Mechaniken des Zeitmanagements und des Stress-/Gesundheits-Managements beinhalten sollte. Dies sollte durch Aufgaben und (Freizeit-)Aktivitäten bewerkstelligt werden.

Konkrete Konsequenzen sollten durch 2 Spielenden abgebildet werden und die Parameter sollten eine visuelle Repräsentation durch Shader finden, wobei diese nur grob beschrieben waren.

In der Umsetzung fanden sich alle Ziele, welche im Exposé festgelegt wurden wieder und zusätzliche Mechaniken wie Tooltips und die Story-Line wurden umgesetzt, da diese das Onboarding neuer Spieler erleichtern und edukative Aspekte des Spiels zugänglicher machen.

Wo gab es Probleme?

- Erstellung von PoCs
- Kommunikation in Audit 3 -> Über Notwendigkeit einer Story
- Merge Konflikte -> Wer arbeitet an welcher Scene
 - Projekt nicht Modular genug
- Lösungen mussten kurzfristig gewechselt werden
- Code nicht „Clean“

Kommen zuletzt noch zu den Problempunkten im Projekt.

Die Erstellung von PoCs war unserer Ansicht nach etwas unintuitiv, daher dauerte dies etwas länger.

Im 3. Audit gab es Missverständnisse bezüglich der Story-Line, da wir unseren Standpunkt, warum die Story Line lediglich ein Zusatz wäre, nicht adäquat vertreten konnten.

Ein Problem, welches vor allem zu Beginn des 4ten Audits auftrat, war dass wir zeitweise Mergekonflikte hatten, da wir das Projekt nicht modular genug bauten und daher immer wieder unabgesprochen Änderungen an der Main-Scene vornahmen. Dies führte zu kleineren Rückschritten, welche aber wieder zeitnahe aufgeholt werden konnten.

Ebenso musste die Lösung zur Darstellung des Textes kurzfristig gewechselt werden, was am Ende hin Zeit kostete.

Ein letztes Problem, welches in der Weiterentwicklung und zum Ende der Entwicklung zum Problem wurde, war dass der Code nicht sauberlich genug implementiert wurde.

Ausblick für potenzielle Weiterentwicklung

- Animation erweitern und verfeinern
- Benutzeroberfläche stilistisch an Raum und Character anpassen
- Story-Line erweitern
- Weitere Freizeitaktivitäten und Arbeitstätigkeiten hinzufügen
- Soundeffekte und Musik
- Sub-Deadlines -> PoC 311
- Weitere Level, für einen graduellen Anstieg der Herausforderung des Zeitmanagements
- Mini-Spiele bei Aktivitäten, und Aufgaben -> Simulation von Aufgaben

Sollte das Projekt weiter entwickelt werden, so gäbe es eine Vielzahl von Punkten, an denen man ansetzen könnte.

Die Animation der Spielfigur könnte vervollständigt, die Benutzeroberfläche an Raum und Spielfigur angepasst, die Story Line erweitert und die Reihe an Aktivitäten und Aufgaben erweitert werden.

Soundeffekte und Musik, sowie der nicht implementierte PoC 311, die Subdeadlines, könnten implementiert werden.

Denkbar wäre es, das Spielgeschehen in Level aufzuteilen, welche einen graduellen Anstieg der Herausforderung des Zeitmanagements bieten könnte.

Die Erstellung von Mini-Spielen könnte die Aktivitäten und Aufgaben ergänzen und so die Attraktivität der Aktivitäten weiter beeinflussen.

Sollte dieses Projekt also weiter geführt werden, könnte der Simulationsaspekt vertieft und die Zugänglichkeit erweitert werden.

Fragen?

Gibt es noch Fragen?

Gibt es noch Fragen?